Development of Git version controller with Rust

Arnau Muñoz

April 6, 2026

Resum – Aquest treball té com a objectiu el disseny i desenvolupament d'un servei de control de versions i gestió de repositoris, amb la idea d'aportar una alternativa independent a plataformes de tercers, sent una proposta gratuïta i de lliure accés. Específicament, es busca una implementació de l'aplicació amb Rust, llenguatge de programació que mitjançant el seu model de "ownership" i el sistema de tipus, ens permet un control exhaustiu i segur de la memòria del programa, generant així robustesa i un rendiment excepcional.

Aquest software busca la centralització de la gestió de versions, assegurant i facilitant la traçabilitat dels canvis en un context fiable i col·laboratiu pel treball distribuït en equip.

Paraules clau – Control de versions, Repositoris, Sistemes distribuïts, Desenvolupament col·laboratiu, Gestió de projectes de Software

Abstract – This project aims to design and develop a version control and repository management service, with the idea of providing an independent alternative to third-party platforms, being a free and open access proposal. Specifically, we are looking for the application implementation to be done in Rust, a programming language that through its ownership model and type system, allows us exhaustive and secure memory control, generating robustness and exceptional performance.

This software seeks the centralization of version management, ensuring and facilitating the traceability of changes with a reliable and collaborative environment for distributed teamwork.

Keywords – Version control, Repositories, Distributed systems, Collaborative development, Software project management

1 Introduction

This project is framed within the current context of distributed software development and the growing need to build robust and scalable systems adapted to collaborative environments. In a technological landscape where established platforms such as GitHub or Bitbucket dominate and are the main choices of an ecosystem of version control and team coordination, the motivation arises to gain an in-depth understanding of the internal mechanisms that make such systems possible.

Beyond replicating an existing solution, this project is driven by a dual motivation. On the one hand, there is a personal motivation to develop a system more specifically tailored for personal use cases, offering greater control over its behaviour, architectural design and future growth. On the other hand, there exists interest in deconstructing and rebuilding from scratch the fundamental principles that enable globally scaled systems to coordinate workspaces, manage concurrent versions and synchronize distributed repositories with multiple collaborators.

Developing this implementation allows me to adopt a critical perspective on architectural decisions, storage models, synchronisation protocols, and user experience design. This process involves not only solving technical challenges but also balancing performance, simplicity, maintainability and decision criteria, shaping a solution aligned with the objectives.

In summary, although this Final Degree Project is based on a concept widely established within the industry, it reinterprets it from an experimental and educational perspective. The objective is not to compete with consolidated platforms, but rather achieve a deep understanding of the systems that underpin modern collaborative work and to

• E-mail of contact: arnaumunozbarrera@gmail.com
• Specialty: Enginyeria del Software
• Tutorized work by: Jordi Casas Roma (Ciències de la Computació)
• Course 2025/26

demonstrate the feasibility of a custom made solution developed from scratch.

2 OBJECTIVES

In order to gain a deeper understanding of how coordination systems operate, particularly in managing simultaneous, collaborative work across shared environments. This section introduces the key objectives of the project.

Each objective presented in the following section is essential and directly contributes to the validation and fulfilment of the overall purpose of the project.

OB1 **Develop a functional system:** a software system that enables the execution of version-control operations in collaborative environments, both in local and remote workspaces.

OB2 **Implement a modular and scalable architecture:** create an architecture that allows future extension and development of the system with new functionalities, ensuring the long-term growth of the tool through an alternative and open-development approach.

OB3 **Develop an analytical interface:** design and build a system focused on analytically visualising repository status through dashboards and visual elements, enabling an intuitive understanding between the user and the system.

OB4 **Provide relevant metrics:** provide the user with relevant metrics and indicative, dynamic visualisations that improve both capability and ease of decision-making regarding project evolution and repository activity.

OB5 **Explore the viability of an alternative solution:** create a proprietary implementation as an alternative to traditional version-control systems, from a modern technical and architectural perspective, for a more user-oriented experience.

3 METHODOLOGY

The implemented methodology for the development of this project was the **DevOps** methodology, which focuses on being constant with the integration process, through the automation of repetitive tasks and switching, as needed, between the phases of development and operation, staying always on the same path to ensure quality and consistent deliverables.

The work strategy was structured into short iterative cycles, combining incremental development practices with continuous integration and validation processes. This was accomplished through the following main actions:

- **Planning and objective definition:** at the start of each cycle, the specified functionalities were decomposed, described and re-organized according to their priority, based on their impact and implementation complexity. This planning ensured a global view of the whole project, adding clear vision and supported coherent progression.

- **Development and Continuous Integration (CI):** each major code change was integrated and verified on a frequent basis, ensuring system stability and reducing the risk of conflicts or accumulated errors. This process enabled early issue detection.

- **Continuous Delivery (CD) and Deploy:** each feature development passed automated build and validation processes to ensure they could be deployed in a controlled and repeatable manner. This approach allowed the system to remain in a functional state at all times.

- **Monitoring and tracking:** project status was continuously monitored with progress and issues alerts. This ongoing oversight facilitated rapid adaptation to changes or new requirements.

Through the application of DevOps practices, the project evolved progressively and was always supported by constant feedback cycles and clear traceability across each development phase. This approach not only improved implementation deadlines but also enhanced the overall quality of the final product and its adaptability to future changes.

As shown in Figure 1 (a) in the ”Appendix”, the project timeline illustrates the general structure of the tasks, as well as their temporal distribution in the Gantt chart, where the overall project planning and the achievement of the main milestones can be observed.

To delve deeper into the process structure, the project is divided into three main work blocks:

- BLK1 **Backend development**, focused on implementing the version control system, managing local and remote repositories and exposing a functional API.

- BLK2 **Frontend development**, oriented towards creating a minimal visual interface that enables the analysis and monitoring of repositories status via dashboard and relevant metrics.

- BLK3 **Transversal and documentation block**, dedicated to planning, validation, testing and results analysis

In addition, a progressive development roadmap is established to structure the system’s evolution.

- **Phase 0:** Market analysis and review of existing solutions.

- **Phase 1:** Architectural design and definition of the conceptual model.

- **Phase 2:** Development of a Minimum Viable Product (MVP).

- **Phase 3:** Optimization of processes, performance and robustness.

- **Phase 4:** Validation, final testing and system evaluation.

4 STATE OF THE ART

Version control systems have emerged as a solution to a widespread problem in software development, becoming a fundamental pillar for code maintenance and management within organisations. In environments where multiple developers work simultaneously on the same project, it is essential to have tools that enable the management of changes, the maintenance of a version history, and the coordination of collaborative work in a secure and efficient manner. This is where **Version Control Systems (VCS)** come into play, providing the necessary mechanisms to maintain a record of the source code, including changes, synchronisations, and the possibility of applying backup or rollback processes.

Over the years, different implementation approaches for these systems have emerged, as **VCS** were not the first solution initially adopted. The earliest systems were centralised version control systems, more commonly known as **Subversion systems (SVN)**, which relied on a connection to a centralised point in order to retrieve changes or submit them for availability to the rest of the team. Although these systems fulfilled the initial objective of simplifying version history control, they still presented several clear limitations, particularly affecting availability, scalability, and the ability to work offline.

With the growth of distributed development and the increasing need for collaborative work across different environments, **Distributed Version Control Systems (DVCS)** gained prominence. From this context emerged **Git**, which has become the most standardised and widely adopted system in the industry to date. Currently, this technology provides the capability to perform actions that go beyond basic version control operations, while also offering high performance and flexibility.

In addition to "pure" version control tools, the current ecosystem of platforms includes additional integrations. Entities such as GitHub, GitLab, and Bitbucket provide the ability to manage remote repositories, perform code reviews, implement largely automatable integration systems, and track issues, among other functionalities. These platforms have significantly transformed the way development teams coordinate their projects, fostering more open and distributed working models.

However, despite the widespread adoption of these tools, most developers who use them tend to regard them as closed solutions, without even considering a deeper exploration of the internal mechanisms that have enabled their extensive adoption and success. Within this context, a personal interest arises in developing a custom implementation of a similar system that allows the exploration and practical understanding of the fundamental principles behind these technologies. This creation is not intended to replace existing solutions, but rather to serve as a challenge aimed at building an experimental environment in which the processes of system design, analysis, and development can be examined.

The project presented in this work is positioned within this line of exploration, proposing the development of a version control system implemented in the **Rust** programming language. This language provides relevant characteristics such as memory safety, high performance, and a type system that supports the development of robust software. Furthermore, the choice of Rust allows for the analysis of how these properties can be applied to the design of infrastructure tools for software development.

In this way, the objective is to achieve a deeper understanding of distributed version control systems by analyzing their technical foundations and evaluating the feasibility of developing a custom implementation.

5 DEVELOPMENT

The development of the project has followed a progressive strategy aimed at consolidating its fundamental pillars from the initial stages. This approach has made it possible to establish a solid foundation upon which to build a robust, scalable implementation aligned with the objectives and requirements defined at the outset of the project.

The following sections provide a more detailed and in-depth view of the most relevant technical aspects. On the one hand, high-level elements are addressed, such as the technological and logical architecture that articulates the different services composing the system. On the other hand, more specific aspects are analysed, including the challenges identified, the emerging requirements, and the technical decisions adopted thus far, thereby offering greater context regarding the design process and the evolution that has led to the current state of the project.

5.1 Architecture and technology stack

The solution proposed for the next phases of the project has been conceived as a modular web architecture, with a clear separation between the user interface, the domain logic of the version control system, backend services, authentication, and persistence. This approach responds to a deliberate design criterion: the functional core of the product must revolve around the intrinsic behaviour of a **Version Control System (VCS)**, preventing auxiliary concerns, such as authentication or database infrastructure management, from prematurely shaping the evolution of the system. At the current stage of development, only the **core** functionality of the **VCS** and an initial foundation of the **REST API** have been implemented. Consequently, the architecture described in this section should be understood as the target structure upon which future extensions of the project will be built.

From a global perspective, the planned architecture follows a client–server model in which the frontend acts as the user interaction layer, while the backend exposes a **REST API** responsible for channelling operations towards the core of the version control system. Unlike a conventional application focused exclusively on **CRUD** operations over business entities, in this case the central element is the **VCS engine** itself, which is responsible for managing repository states, file modifications, commits, branches, and synchronisation. The API is therefore not conceived as the heart of the system, but rather as an access interface to the domain, allowing the main logic to remain decoupled and facilitating its future reuse from other clients, such as a more advanced web interface or even a dedicated **CLI**, as shown in Figure 1 (b) in the "Appendix".

At the presentation layer, a frontend based on **React**, supported by a prior interface design in **Figma**, has been envisaged with the aim of building a modular, reactive, and easily extensible application. The choice of React is consistent with the need to represent changing repository states, comparative views, metrics, and branch diagrams, all of this with dynamic updates and smooth navigation. Furthermore, the component-based approach is particularly well suited to an interface that is expected to increase in complexity as new repository visualisation and analysis capabilities are incorporated.

The functionality planned for this interface is not limited to exposing the basic operations of the system, but rather contemplates an evolution towards a visual environment for repository analysis and interaction. As reflected in the use case diagrams, the user layer has been designed to support actions such as repository creation and initialisation, repository status inspection, branch switching, file content visualisation, file comparison, staging content for commit, committing changes, and remote synchronisation through push and pull operations. On top of this operational basis, additional value-oriented functionalities are also projected, such as commit history visualisation, branch diagrams, comparative metrics, filter application, graph type selection, and change alert systems. This approach demonstrates that the architecture is not only intended to replicate version control operations, but also to provide a layer of observability and visual understanding of the repository state, as shown in Figure 2 (b) in the "Appendix".

One of the most relevant aspects of the architecture is the explicit definition of repository and file state flows. These diagrams do not merely fulfil a documentary function, but rather serve as the conceptual basis for structuring the internal logic of the **VCS**.

Formalising these transitions is essential in order to maintain semantic coherence throughout the system, especially in later phases where both the API and the interface will be required to represent the behaviour of the versioning engine in a consistent manner. In this way, the architecture becomes aligned with an explicit state model, rather than with a simple collection of disconnected operations, as shown in Figure 2 (a) in the "Appendix".

With regard to the backend, a service layer implemented in **Rust** has been defined, connected to the **VCS** core and exposed externally through a **REST API**. The choice of this structure allows the backend to act as an intermediary between client actions and the internal engine operations, encapsulating validations, response serialisation, error handling, and persistence access. At this stage of the project, this **REST** layer is still in an initial state; however, its design is oriented towards incremental growth, progressively incorporating endpoints for repository operations. The objective is not to transfer version control logic to the frontend, but rather to concentrate it within a robust domain layer and to maintain the interface as a decoupled consumer.

Within this context, the selection of already functional systems such as **Supabase**, which provides a persistence and data services solution, is particularly appropriate for the early and intermediate phases of the project. Supabase offers a managed **Postgres** database and a platform designed to accelerate development, with automatically generated APIs and complementary services ready for integration, thus significantly reducing the infrastructure burden required to begin iterative development. In addition, it currently provides a free-tier plan and documentation oriented towards rapid adoption, which is especially valuable in a project where the objective is to maximise the time devoted to the development of the functional domain rather than to platform administration.

The choice of **Supabase** is therefore not based solely on economic considerations, but also on strategic ones. By externalising the complexity associated with provisioning and maintaining a managed database, the project can focus on properly defining the model of commits, trees, branches, and states, which constitutes the true differentiating value of the solution. In other words, effort is not invested in non-priority concerns, such as deploying a database cluster, performing initial infrastructure hardening, or manually building auxiliary utilities, and priority is instead given to the development of the **VCS** engine, its API, and its operational semantics. This decision is coherent with an incremental approach: first consolidating the behaviour of the system and only assuming greater operational complexity when the product genuinely requires it.

Similarly, the incorporation of **Clerk** for authentication and session management is also justified by criteria of simplicity, speed of integration, and focus on the main domain. Clerk provides prebuilt components, **SDKs** for **React**, and authentication utilities that greatly facilitate the incorporation of login, registration, session management, and token acquisition, without the need to implement a complete identity subsystem from scratch. Likewise, it currently maintains a free-tier plan and a proposal strongly oriented towards developer experience, making it a particularly suitable option when authentication does not constitute the primary objective of the project.

Another noteworthy aspect is that the future architecture preserves a clear separation between the **core** capabilities of the system and complementary observability or analytics functionalities. The existence of modules oriented towards metric visualisation, branch comparison, change alerts, or evolution diagrams does not alter the central role of the versioning engine, but rather builds upon it. This distinction is important because it enables the development process to be planned in layers: first consolidating the fundamental operations of the repository and of versioned objects, then stabilising the API, and finally building upon that foundation visualisation and user assistance tools with greater added value.

Taken as a whole, the architecture envisaged for the next phases of the project responds to a criterion of progressive, modular, and domain-centred growth. The solution relies on a modern frontend for interaction and visualisation, a **REST API** as the exposure layer, a backend structured around the **VCS** core, flexible and evolvable persistence, and external services, such as **Supabase** and **Clerk**, selected precisely because they allow those parts of the system that, while necessary, do not constitute the principal focus of the work to be resolved in a simple and cost-effective manner. In this way, the project avoids dispersing effort across peripheral components and directs most of the development towards the construction of a functional, consistent, and scalable version control system.

5.2 Challenges

During the development of the project, several technical challenges were identified that required specific solutions and, in some cases, a partial reconsideration of the initial approach. The adoption of **Rust as the primary programming language** introduced a significant learning curve, particularly due to its ownership model and strict type system. This required an initial phase of adaptation and experimentation before reaching a minimum level of proficiency that allowed for the development of valid components.

At the same time, building **a personal implementation of a distributed version control system** demanded a deep understanding of Git's internal mechanisms. This involved analysing concepts such as object-based storage, reference handling, and state management, and then reinterpreting them into a custom-designed system. Closely related to this, the design of **the storage system** itself introduced additional complexity, as it required addressing object serialization, disk persistence, and data compression, while ensuring consistency, integrity, and efficiency.

Furthermore, the need to support communication between repositories led to the design of **a foundational HTTP API and a distributed system architecture**. This implied defining clear communication structures and anticipating future scalability and extensibility requirements from the early stages of development.

As a key technical adjustment, the project also adopted **a parallel development approach**, in which multiple modules, including the CLI functionalities, the backend-oriented API, and the frontend components, are being developed simultaneously within a unified working environment. This decision facilitated coordination, improved interoperability between components, and enabled a more agile and coherent evolution of the system.

Overall, these challenges not only contributed to the technical consolidation of the project but also fostered a progressive refinement of architectural decisions, allowing the solution to adapt continuously throughout the development process.

5.3 Project state

The project is progressing according to the established schedule, maintaining steady development while preserving a reasonable margin to accommodate potential unforeseen issues. At this stage, the key functionalities defined thus far have been successfully delivered.

The results achieved to date are outlined in the following section, providing a clear overview of the project's overall progress and its iterative evolution across successive development cycles.

6 Results

The objectives defined at the beginning of the project have, up to this point, been satisfactorily achieved in alignment with the planned schedule. Through the adoption of an iterative planning approach and thorough dependency analysis, which have enabled greater technical adaptability, the following objectives have been successfully achieved:

- **Definition of the Storage System**: This includes the initialisation of the data management folder, object serialisation processes, disk persistence mechanisms, data compression, and the management and logical handling of file references and states. This system ensures efficient data storage, retrieval, and integrity across operations.

- **Development of the Command Line Interface (CLI)**: This involves the implementation of core functional commands along with their respective variations, in addition to supplementary and utility commands designed to simplify usage and streamline the overall interaction with the system. The CLI provides a flexible and accessible way to execute system operations.

- **Database Design**: The structure and organisation of the database have been defined to support the requirements of the system, allowing efficient data storage, consistency, and scalability. This design lays the foundation for reliable data management and future extensibility.

- **Logical Design of the Basic HTTP API**: A foundational structure for the HTTP API has been established, defining endpoints, request-response behaviour, and interaction logic. This allows external systems or clients to communicate with the application in a standardized and scalable manner.

To conclude, the presented objectives are being progressively and successfully accomplished, meeting the initial requirements of the project and establishing a solid foundation for subsequent iterations.

Use of generative AI

In this work, the generative artificial intelligence tool, ChatGPT-o4 mini, has been used for translation support. The use of this tool has been applied in the following parts of the work: summary and methodology.

All content generated with the support of AI tools has been reviewed, verified, and edited by the author prior to its inclusion in the final version of the document.

The final content is a faithful representation of the work carried out by the author and of their intellectual contributions.

The author declares under their responsibility that:

· The content of the work is truthful, complete, and accurate;

· They have made transparent use of generative AI tools;

· And they assume full responsibility for the authorship and content of this work.

References

[1] Jira "Creador de diagramas de Gantt", [Online] Available at: https://support.atlassian.com/jira-product-discovery/docs/create-a-timeline-view/ [Revised: 20-feb-2026]

[2] DevOps ”DevOps”, [Online] Available at: https://es.wikipedia.org/wiki/DevOps [Revised: 26-feb-2026]

[3] API in Rust ”Creating a REST API in Rust”, [Online] Available at: https://arshsharma.com/posts/rust-api/ [Revised: 23-mar-2026]

[4] Supabase ”Getting Started”, [Online] Available at: https://supabase.com/docs/guides/getting-started [Revised: 24-mar-2026]

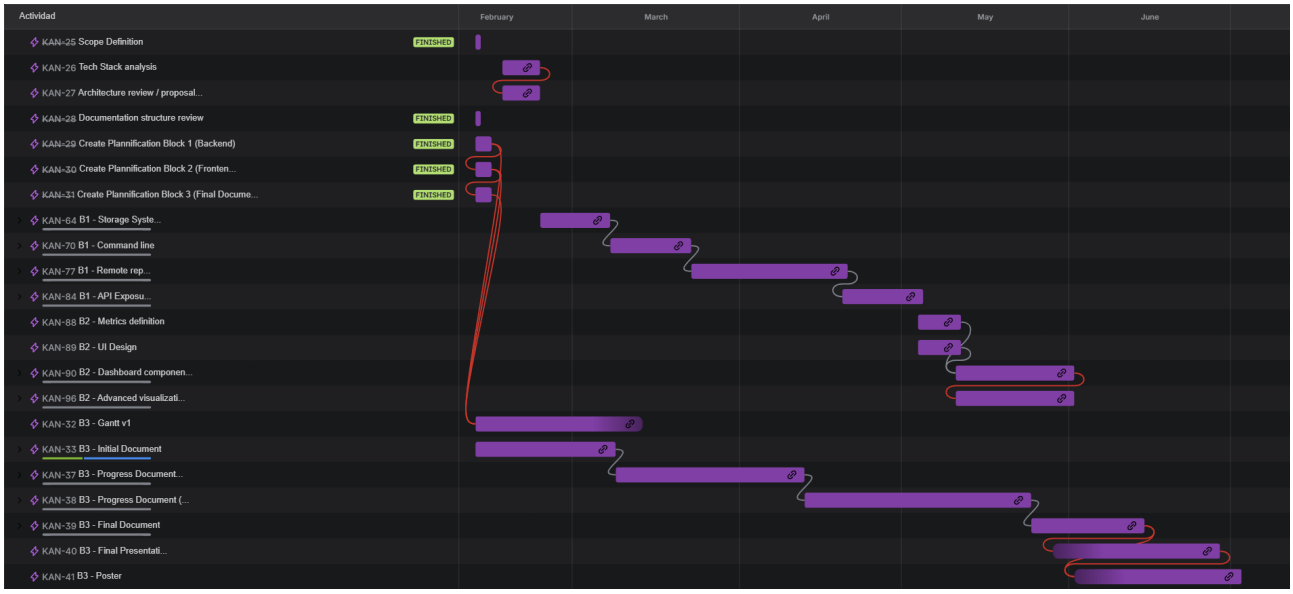
[5] Clerk ”Quickstarts”, [Online] Available at: https://clerk.com/docs/getting-started/quickstart/overview [Revised: 02-may-2026]

APPENDIX

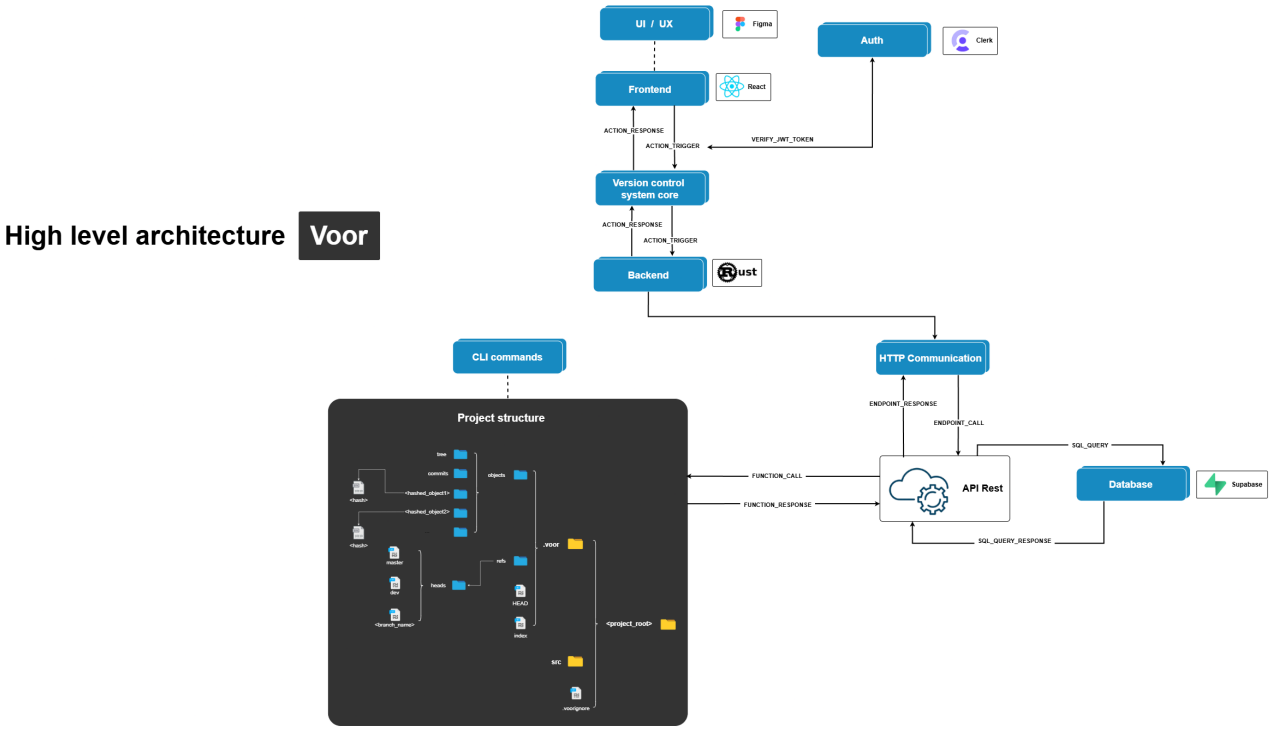
A.1 Methodology

This section presents project planning designed to track deadlines, tasks, and deliverables using Jira as a management tool. The objective is to ensure effective coordination, monitoring, and control throughout the project lifecycle.

The following images provide an overview of the overall organisation of tasks. They illustrate the main development areas (Backend, Frontend and Documentation), each structured with their respective timelines, milestones and deadlines, represented through a Gantt chart.

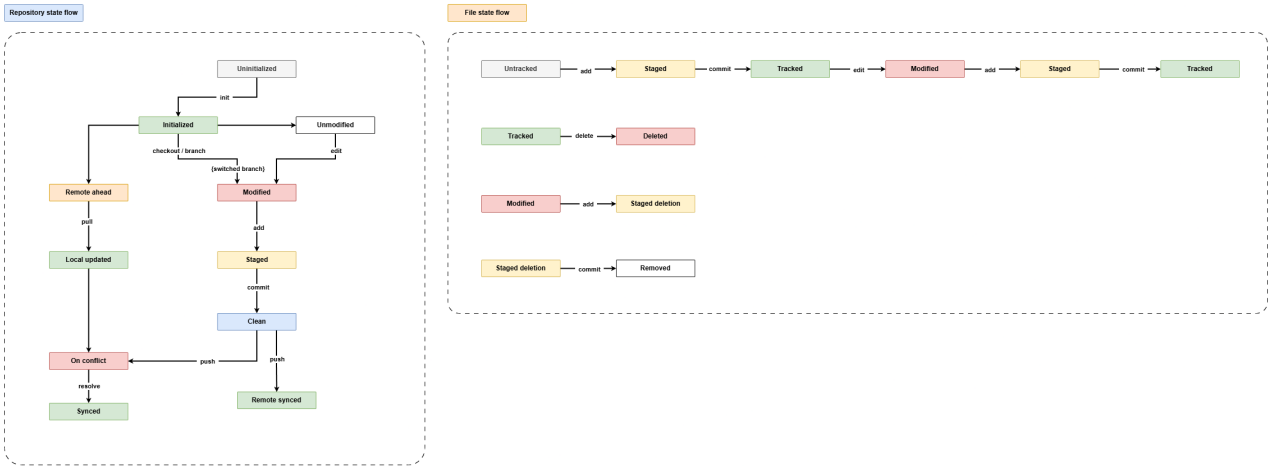


(a) Gantt diagram showing the project timeline and task distribution.

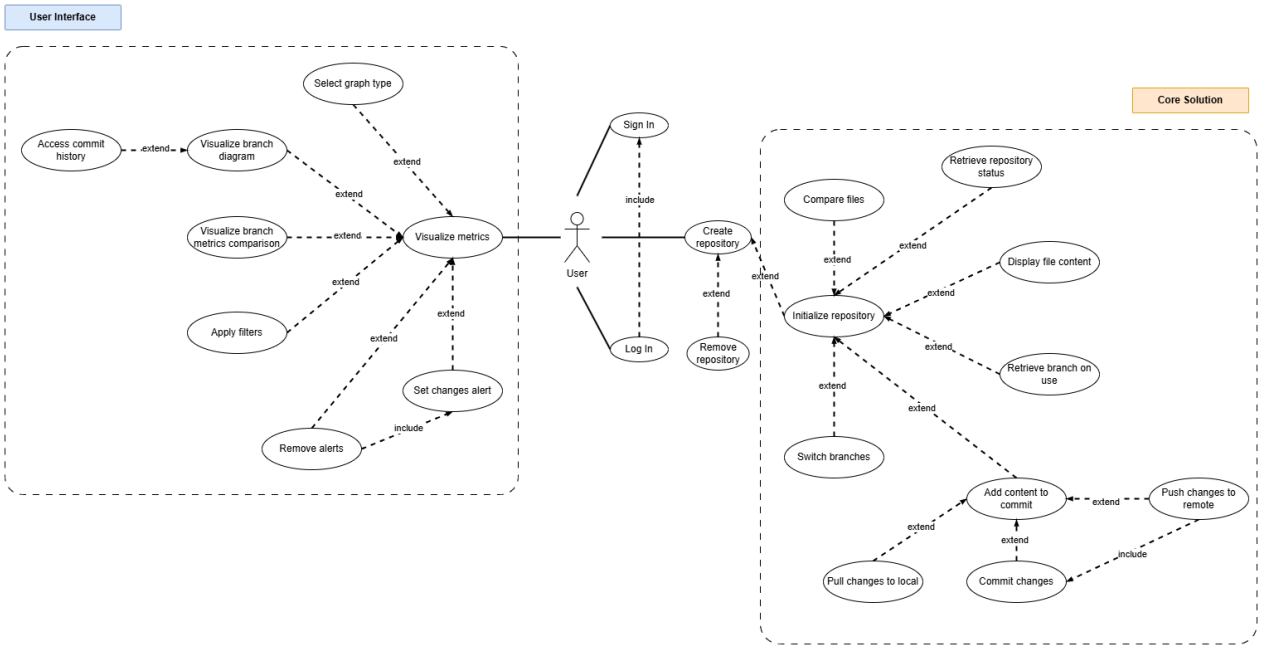


(b) High-level system architecture describing the main components and interactions.

Fig. 1: Project architecture overview.



(a) Repository and file state flow illustrating internal data transitions.



(b) User use case actions overview.

Fig. 2: User and file state flow overview.